Terminal propagation

Pads and fixed I/O pin nodes to sides before free cells move

The idea

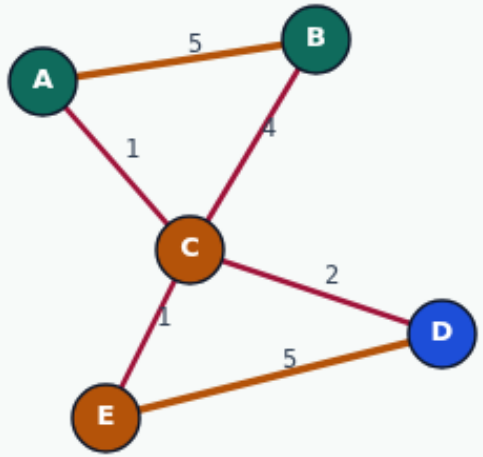
- Fixed terminals never flip
- Their neighbors inherit a pull toward the terminal’s part
- Ignoring terminals produces pretty cuts that violate the floorplan interface
- <!-- algorithm-walkthrough -->

Fix pads A and E to opposite sides

TERMINAL PROPAGATION

Fix pads A and E to opposite sides

Step 1 / 5 · fixed-terminals · module03-03-terminal-propagation



Explanation

I/O terminals are pinned: A → side 0, E → side 1. Free nodes B, C, D propagate by taking the side of the strongest neighboring fixed/assigned node.

- Terminals never flip
- Free nodes vote by edge weight
- Iterate until stable

```
fixed: A=0, E=1
free: B, C, D
```

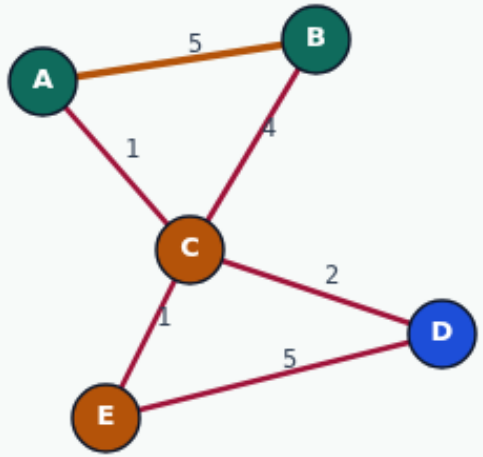
Fix pads A and E to opposite sides

B joins A (w=5)

TERMINAL PROPAGATION

B joins A (w=5)

Step 2 / 5 · b-joins-a · module03-03-terminal-propagation



Explanation

B hears A strongly on side 0 (A–B weight 5) versus weak pull from E. B adopts side 0 immediately.

- vote(side) = Σ w to neighbors on that side
- Heavy A–B affinity wins
- Fixed A anchors the ABC community

```
B → side 0
A–B internal
```

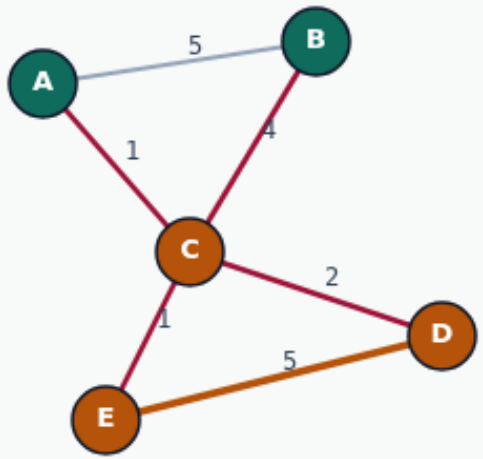
B joins A (w=5)

D joins E (w=5)

TERMINAL PROPAGATION

D joins E (w=5)

Step 3 / 5 · d-joins-e · module03-03-terminal-propagation



Explanation

D hears E on side 1 (D–E weight 5). D adopts side 1, forming the DE block opposite ABC.

- Symmetric to B joining A
- Fixed E anchors DE
- Two communities emerging

```
D → side 1
D–E internal
```

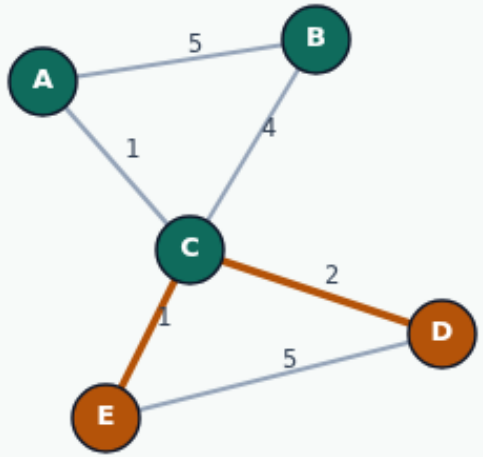
D joins E (w=5)

C bridges by neighbor vote

TERMINAL PROPAGATION

C bridges by neighbor vote

Step 4 / 5 · c-bridge · module03-03-terminal-propagation



Explanation

C connects to both communities. Neighbor vote sums favor joining A/B side (w=4+1 vs w=2+1). C lands on side 0 with A,B.

- Bridge node follows stronger pull
- Cut edges: C–D, C–E only
- cutsize = 3

```
C → side 0
parts: ABC|DE
cutsize: 3
```

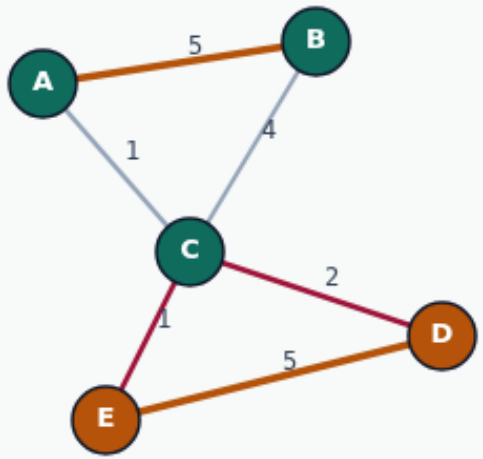
C bridges by neighbor vote

Fixed I/O drives partition

TERMINAL PROPAGATION

Fixed I/O drives partition

Step 5 / 5 · takeaway · module03-03-terminal-propagation



Explanation

Terminal propagation converges in 2 iterations on this graph. Real designs pin hundreds of pads — free logic follows connectivity to terminals.

- A/E terminals → ABC|DE
- Same golden as spectral/grow
- Cheap seed before FM refine

```
iters: 2
cutsize: 3
```

Fixed I/O drives partition

Browser lab track

- In the browser lab track, open the **terminal-propagation** lab from the tools shelf
- Load the starter graph, run the algorithm once
- Work the challenges that lock the goldens

Implement track

- In the implement track, open this module’s examples and the course `common/` solvers
- Parse the tiny graph, run the algorithm with a deterministic seed
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps
- For multilevel flows, verify coarsening before you blame the refiner

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you’re ready, take the short quiz, then continue to the next module